

GPU & Graphics & Compute APIs — Entry Point

A concise, organized entry-point for research into graphics and compute APIs. Each entry has a 5–7 line summary, key uses, and short notes on ecosystem/compatibility. Use this document as a map for deeper reading.

Table of contents

1. OpenGL
2. OpenGL ES
3. WebGL
4. Vulkan
5. Direct3D (DirectX)
6. DirectCompute
7. DirectX Raytracing (DXR)
8. CUDA
9. OpenCL
10. Metal
11. WebGPU
12. Mantle (historic)
13. ROCm / HIP
14. SYCL / oneAPI / Level Zero
15. OptiX
16. OpenVX
17. GLSL, HLSL, SPIR-V (shading languages & IR)
18. ANGLE / MoltenVK (compat layers)
19. NVAPI / AMD AGS (vendor driver APIs)
20. NNAPI / Core ML / ML inference APIs
21. RenderScript (deprecated) & mobile compute
22. XR Graphics APIs (OpenXR + integrations)
23. Scene graph & higher-level engines (brief)

1. OpenGL

- The Open Graphics Library is a long-standing, cross-platform, immediate-mode-style graphics API primarily focused on rasterized 2D/3D rendering.
- It exposes a programmable pipeline (vertex + fragment shaders, and later geometry/tessellation) and low-level buffer/texture management.
- Historically ubiquitous on Windows, Linux and older macOS; great for learning GPU pipeline fundamentals.
- Drivers translate OpenGL calls to GPU instructions; the API abstracts hardware but does not guarantee uniform performance across vendors.
- Modern alternatives (Vulkan/Metal/Direct3D12) provide lower-level control and explicit multi-threading; OpenGL remains useful for portability and education.

2. OpenGL ES

- A trimmed, embedded variant of OpenGL designed for mobile, embedded systems, and consoles with constrained resources.
- Supports a subset of OpenGL features and mobile-friendly shader/texture formats; versions evolve to include more modern pipeline features.
- Used widely on Android, many embedded devices, and older iOS support (historically until Apple pushed Metal).
- Implementations must be lightweight and highly optimized for power and thermal limits of mobile GPUs.
- If targeting mobile platforms, learn both OpenGL ES and its modern successors (Vulkan on mobile).

3. WebGL

- A JavaScript binding of OpenGL ES for the web, enabling hardware-accelerated 2D/3D graphics inside browsers without plugins.
- WebGL 1.0 maps closely to OpenGL ES 2.0; WebGL 2.0 aligns with OpenGL ES 3.0 features (multiple render targets, etc.).
- Security and sandboxing considerations restrict direct GPU control; browser vendors provide standardized behavior.
- Great for interactive visualizations, web games, and delivering demos without installing native code.
- WebGPU is emerging as a modern successor with lower-level control and compute support.

4. Vulkan

- A modern, low-level, cross-platform graphics and compute API from Khronos that gives explicit control over GPU resources and multithreading.
- Designed for minimal driver overhead; applications manage memory, synchronization, pipelines, and command submission explicitly.
- Superior for high-performance engines where predictability and multi-core CPU scaling matter (AAA engines, real-time renderers).
- Uses SPIR-V as an intermediate shader representation, decoupling shader languages and enabling validation/optimization.
- Steeper learning curve than OpenGL, but offers better performance and portability across desktop and mobile (via Vulkan ES).

5. Direct3D (DirectX)

- Microsoft's flagship graphics API (part of DirectX) used across Windows and Xbox platforms; Direct3D 12 is the modern, low-level variant.
- Like Vulkan, D3D12 offers explicit resource control, CPU multithreading efficiency, and low driver overhead for high-performance games.
- Tightly integrated with Windows and Xbox OS features, drivers, and tooling (PIX, Visual Studio graphics debugging).
- Historically dominant in AAA game development on Windows/Xbox; many engines expose a Direct3D backend.
- Has language bindings (HLSL) and ecosystem features like DirectML and DirectX Raytracing (DXR).

6. DirectCompute

- A compute-shader API that is part of Direct3D/DirectX; it lets developers run general-purpose computation on GPU via compute shaders.
- Designed for Windows platforms and integrated into the DirectX ecosystem for workloads like physics, postprocessing and GPGPU tasks.
- Uses HLSL (shader language) for compute kernels, letting graphics shaders and compute shaders share tooling and memory resources.
- Less cross-platform than OpenCL/CUDA; best when your app is already Windows-centric or tightly integrated with Direct3D pipelines.
- Useful for game engines and Windows apps that require GPU-accelerated compute without leaving the DirectX stack.

7. DirectX Raytracing (DXR)

- An extension to Direct3D that provides GPU-accelerated ray tracing primitives, acceleration structures (BLAS/TLAS), and ray generation/hit shaders.
- Enables realistic lighting, reflections, and shadows in real-time applications; tightly integrated with D3D and modern GPUs that provide RT cores.
- Works as a high-level API for ray traversal while leaving lower-level scheduling and memory management to the app/driver.
- Adopted rapidly in games and engines for hybrid rasterization + ray tracing rendering workflows.
- Conceptually comparable to vendor APIs like NVIDIA OptiX (for ray workloads) but standardized inside Direct3D.

8. CUDA

- NVIDIA's proprietary parallel computing platform and programming model for NVIDIA GPUs; offers fine-grained control and high performance.
- Uses an extended C/C++ language with kernels, streams, and specialized libraries (cuBLAS, cuDNN, cuFFT) for ML, HPC, and scientific computing.
- Dominant in machine learning and HPC because of mature tools, optimized libraries, and broad GPU hardware deployment.
- Vendor-locked: CUDA runs only on NVIDIA GPUs, but it's often the fastest path for production ML workloads on NVIDIA hardware.
- Learning CUDA is extremely valuable if your research or projects will target NVIDIA GPUs or use CUDA-accelerated libraries.

9. OpenCL

- The Open Computing Language provides a vendor-neutral framework for writing programs that execute across CPUs, GPUs, FPGAs and other accelerators.
- Exposes kernels, command queues, and device/host memory management; kernels are portable C-like code that vendors compile for hardware.
- Cross-vendor portability is a major strength but performance portability (optimal tuning across devices) can be challenging.
- Wide use in heterogeneous compute tasks: video processing, scientific simulations, and some ML workloads (less dominant than CUDA in ML).

- Good for proving algorithms across multiple hardware types or when vendor lock-in is unacceptable.

10. Metal

- Apple's low-level graphics and compute API for iOS, macOS, and tvOS, combining rasterization, compute shaders, and GPU-side ray tracing over one stack.
- Designed for high performance and energy efficiency on Apple silicon; replaces OpenGL/OpenCL on Apple platforms.
- Provides tight OS and driver integration, mature tooling (Metal Performance Shaders) and optimized libraries for ML and graphics.
- If targeting Apple devices, Metal is the practical production API—Vulkan/Direct3D are not native on Apple.
- MoltenVK can map Vulkan to Metal for portability, but native Metal yields the best performance and ecosystem fit.

11. WebGPU

- A modern web standard designed to replace/augment WebGL by exposing more explicit, low-level GPU control and compute on the web.
- Maps to native APIs (Vulkan/Metal/Direct3D) under the hood and supports both graphics and compute workloads with a safer, web-friendly model.
- Offers better performance and feature parity with native low-level APIs while preserving browser sandboxing and security.
- Ideal for complex web-based games, ML inference in-browser, and advanced web visualizations that need compute shaders.
- Still relatively new—adoption is growing and it's a recommended target for forward-looking web GPU work.

12. Mantle (historic)

- AMD's low-level graphics API created to reduce driver overhead and demonstrate explicit, low-level control.
- Mantle heavily inspired Vulkan's design; many concepts and lessons from Mantle fed into Khronos' Vulkan specification.
- Mantle itself is deprecated and not used directly, but it's important historically when studying modern API evolution.
- Understanding Mantle explains why explicit control and reduced driver responsibility matter for performance.
- Treat Mantle as an academic ancestor to Vulkan rather than a production API to learn today.

13. ROCm / HIP

- ROCm (Radeon Open Compute) is AMD's open software stack for GPU compute; HIP is a CUDA-like portability layer for running CUDA code on AMD hardware.
- HIP provides a pathway to port CUDA kernels to run on AMD GPUs with minimal source changes; ROCm supplies drivers and optimized libraries.
- Growing in HPC and open-source ML tooling—useful if you want to support AMD hardware or build vendor-agnostic code.

- The ROCm ecosystem includes libraries for ML, math, and profiling similar to NVIDIA's CUDA ecosystem.
- Key for multi-vendor compute portability and avoiding CUDA-only lock-in when targeting diverse clusters.

14. SYCL / oneAPI / Level Zero

- SYCL is a higher-level C++ abstraction (from Khronos) for heterogeneous compute, enabling single-source kernel programming using modern C++.
- Intel's oneAPI and DPC++ build on SYCL ideas; Level Zero is a low-level runtime that exposes direct hardware control on Intel accelerators.
- oneAPI aims for cross-architecture portability (CPUs, GPUs, FPGAs) and provides libraries for domains like AI and HPC.
- SYCL offers a friendlier, modern-C++ developer experience compared to raw OpenCL while still targeting heterogeneous devices.
- If you prefer C++ idioms and cross-architecture portability, SYCL/oneAPI are essential to study.

15. OptiX

- NVIDIA's high-level ray tracing engine and API tailored to accelerate ray tracing algorithms on NVIDIA GPUs.
- Provides ray generation, traversal, intersection, and shading constructs; optimized to use NVIDIA hardware features like RT cores.
- Widely used in offline rendering, real-time ray-acceleration pipelines, and research into physically accurate rendering.
- Sits at a higher abstraction than raw Vulkan/DirectX ray tracing primitives, offering developer productivity and performance.
- Vendor-specific (NVIDIA) but highly performant where NVIDIA hardware is the target.

16. OpenVX

- An open standard (Khronos) focused on accelerating computer vision pipelines on heterogeneous hardware.
- Defines a graph-based API for vision operators (filters, feature detectors, transforms) enabling optimizations and hardware acceleration.
- Useful for embedded vision (cameras, robotics, automotive) where standardized, efficient vision primitives matter.
- Unlike raw compute APIs, OpenVX provides domain-specific building blocks that are easier to optimize across devices.
- Good to know if your research touches applied computer vision on constrained or embedded devices.

17. GLSL, HLSL, SPIR-V (shading languages & IR)

- GLSL (OpenGL Shading Language) and HLSL (High-Level Shading Language) are source-level languages for writing GPU shaders.
- SPIR-V is a binary intermediate representation used by Vulkan (and optionally by other APIs) to decouple high-level languages from drivers.

- Learning shader languages is essential: vertex/fragment (pixel) shaders, geometry, tessellation, and compute shaders all rely on these.
- Shaders control per-vertex and per-pixel work and are where most GPU-parallel logic for graphics and compute lives.
- Toolchains convert GLSL/HLSL or modern front-ends (like glslang or DXC) into SPIR-V for robust cross-driver deployment.

18. ANGLE / MoltenVK (compatibility layers)

- ANGLE translates OpenGL ES API calls to native backends (Direct3D, Vulkan, Metal)—widely used in browsers to support WebGL.
- MoltenVK maps Vulkan calls to Apple’s Metal, enabling Vulkan-based apps to run on macOS/iOS with a compatibility layer.
- These layers are practical when you want cross-platform portability without rewriting the renderer for each native API.
- They impose some overhead and compatibility caveats but save significant development effort.
- Useful in research to prototype on multiple platforms quickly and evaluate API differences.

19. NVAPI / AMD AGS (vendor driver & control APIs)

- NVAPI (NVIDIA) and AMD AGS expose vendor-specific driver-level controls, GPU info, and extended features not present in standard APIs.
- Useful for querying GPU topology, controlling GPU-specific settings, or accessing vendor-only optimizations and debugging features.
- Not portable—useful when you need vendor features (proprietary extensions, advanced profiling, SLI/CrossFire controls).
- Often used in performance tuning, driver-level diagnostics, and vendor-specific feature toggles in production software.
- Treat these as advanced tools for platform-specific optimization.

20. NNAPI / Core ML / ML inference APIs

- Platform-level ML inference APIs: Android NNAPI, Apple Core ML, and vendor runtimes let you run trained models on-device with hardware acceleration.
- They hide device-specific acceleration (GPU, NPU, DSP) behind a standardized interface for mobile/embedded deployments.
- Not full compute APIs but crucial when your research involves deploying ML models efficiently to edge devices.
- Integrates with lower-level APIs (Metal, Vulkan, OpenCL) behind-the-scenes to use the best available accelerator.
- Learn these if you plan to move models from research prototypes into mobile/embedded products.

21. RenderScript (deprecated) & mobile compute notes

- RenderScript was Android’s previous approach for on-device parallel compute (now deprecated in favor of NNAPI and Vulkan compute).
- Historical relevance: shows the industry trend from higher-level mobile compute layers to unified, low-level APIs (Vulkan, Metal) and ML-focused runtimes.

- If targeting Android today, prefer Vulkan compute, NNAPI, or frameworks that emit Vulkan/Metal code.
- Mobile compute requires attention to power, memory, and specialized hardware (NPUs) that differ from desktop GPUs.
- Mobile profiling tools and energy-aware design are essential for production-grade mobile compute work.

22. OpenXR + XR-related graphics integration

- OpenXR is a Khronos standard for virtual reality (VR) and augmented reality (AR) runtimes and device interoperability.
- It defines how applications enumerate devices, render frames, and submit layers—while integrating with graphics APIs like Vulkan, OpenGL, or Direct3D.
- Important for building VR/AR applications where low-latency, synchronized rendering and device tracking are critical.
- OpenXR separates XR runtime mechanics from the rendering API, letting you choose the best GPU API per platform.
- If your research touches immersive graphics, OpenXR + Vulkan/Direct3D is a modern stack to learn.

23. Scene-graph & higher-level engines (brief)

- Above these low-level APIs sit engines and scene graphs (Unity, Unreal, OGRE, three.js, bgfx) that abstract GPU details and accelerate development.
- Useful for rapid prototyping and focusing on algorithms rather than driver/memory management.
- Researchers often use engines to validate ideas quickly, then reimplement performance-critical parts with low-level APIs.
- Know when to use an engine (productivity) vs when to dive into Vulkan/Direct3D/CUDA (control & performance).
- Many engines provide multiple backends—study them to understand real-world API mappings.

How to use this document

- Start with **OpenGL / GLSL** for pipeline fundamentals and simple shaders.
- Move to **Vulkan / SPIR-V** or **Direct3D12** when you need explicit control and multi-threaded performance.
- For compute-heavy research, learn **CUDA** (NVIDIA-dominant) and **OpenCL / SYCL / ROCm** for cross-vendor work.
- For mobile/Apple-specific targets, prioritize **Metal** (and MoltenVK for cross-porting).
- Use **WebGL / WebGPU** for web demos and accessibility.

If you want this exported as a PDF/Markdown file or want me to expand any entry into a multi-section deep-dive (history, code samples, tooling, profiling, libraries, and research papers), tell me which items to expand and I'll extend the document accordingly.